Recommender Systems - Course Project Report

Cumulative report template - Parts 1, 2, and 3

Team	PILIJA
Members	Piitu Varpuluoma, Liisa Piirainen, Jaan Barlow
For Part 3 track: select one	A / B / C
GitHub repository link	https://github.com/lispetti/recsys-movielens

1. Part 1 - Data, Protocol and Baselines

1.1 Data characterization

MovieLens 1M contains 1,000,209 ratings from 6,040 users on 3,706 of the 3,883 catalogue movies; 177 have no ratings. The matrix is 4.47% filled over rated movies (Lecture 1.1, slide 6), and every user has at least 20 ratings.

Ratings per movie have a long tail (Figure 1; Lecture 5.1, slide 5), with a median of 123.5 and a mean of 269.9; ratings per user have a median of 96 and a mean of 165.6. The top 10% of rated movies receive 44.5% of ratings (Gini 0.63), while 663 rated movies have fewer than 20. This concentration favors popularity baselines (Lecture 1.1, slide 9). Ratings skew positive: 1–5 make up 5.6%, 10.8%, 26.1%, 34.9% and 22.6%, with a mean of 3.58.

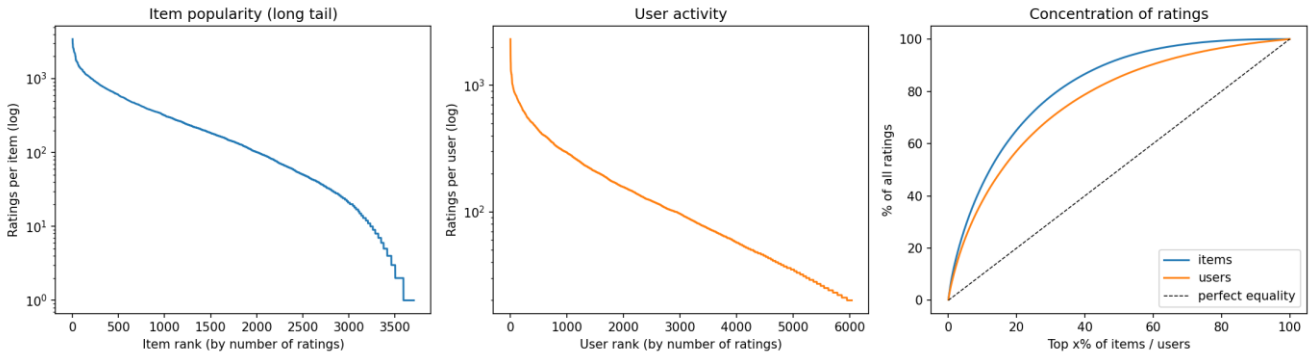


Figure 1. Ratings per movie (left) and per user (centre), ranked from most to fewest, on a log scale. Right: cumulative share of all ratings held by the top x% of rated movies and of users. Computed on the full dataset, before splitting.

Activity is very bursty (Figure 2). The data spans April 2000 to February 2003, but 904,757 ratings were entered in 2000, as the dataset contains users who joined that year (README). 58.7% of users rated on a single calendar day, and 77.4% of ratings share their exact second with another rating by the same user, so timestamps do not show viewing order. A cut by rating counts therefore places validation in 2–29 December 2000, where 611 of 1,134 active users have no training history, against 29 of 1,209 in test (see 1.3).

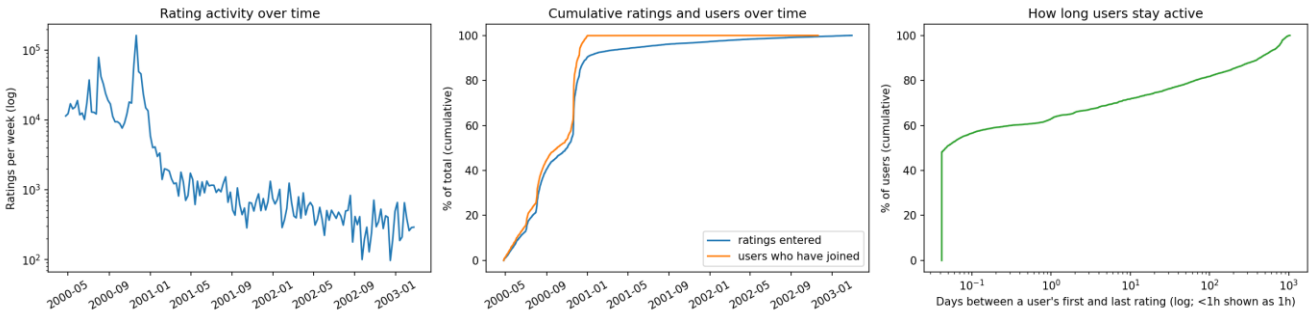


Figure 2. Left: ratings entered per week, on a log scale. Centre: cumulative share of ratings entered and of users who have given their first rating. Right: cumulative share of users by days between their first and last rating, on a log scale, with gaps under one hour shown as one hour. Computed on the full dataset, before splitting.

1.2 Preprocessing and feedback treatment

We apply no filtering, so no rows are removed and all 1,000,209 ratings are used. The dataset already contains only users with at least 20 ratings. Low-count movies are kept, in line with ranking against the full catalogue (Lecture 3.1, slide 5).

Ratings are explicit, so low ratings show dislike (Lecture 1.1, slide 7) and we do not treat every rating as positive. A held-out rating counts as relevant if it is 4 or higher, and relevance is binary in all metrics, including NDCG. With a mean rating of 3.58, a 3 is below average. Under this threshold, 54.4% of the evaluated test users' held-out ratings are relevant, and 21 test users and 31 validation users are excluded because they have no relevant item.

1.3 Train/validation/test split

We use one global temporal split, with the same cut points for all users, because it mirrors deployment: the model always predicts forward in time (Lecture 3.1, slide 4). Ratings before 2 December 2000, 14:52:18 UTC are training data (800,164 ratings). Ratings from then until 29 December 2000, 23:43:34 UTC are validation (100,024), and the rest are test (100,021). A rating with exactly the same timestamp as a cut goes to the later part, so ratings from the same second are never separated by a cut. The split is deterministic.

The cuts are the 80% and 90% points of the rating timestamps, not calendar dates. Activity is so uneven that a calendar split would leave little test data: only 24,046 ratings come from 2002 and 3,348 from 2003. Models are tuned by fitting on train and scoring on validation, then refit on train+val and evaluated once on test (Lecture 3.1, slide 10).

The burst of activity shapes who is evaluated. Validation covers four weeks with many new users, while test covers more than two years of mostly returning users, so 492 users are evaluated on validation and 1,159 on test (see 1.4). Test users are heavier than average, with a median of 150 train+val ratings against 86 for all users. Their test ratings also lean less towards popular movies: 35.5% fall on the 10% most-rated movies, against 45.0% in train+val. Our results therefore describe returning, heavier users, not typical users. Validation and test also differ from each other (a median of 13 against 39 held-out ratings per evaluated user), so settings tuned on validation may not be the best on test.

1.4 Evaluation protocol

Candidate set. Every model ranks the full catalogue (3,883 movies), minus the movies the user already rated in the history data. Ranking the full catalogue avoids sampled-candidate metrics, which can change model rankings (Lecture 3.1, slide 5). Held-out ratings on movies with no history stay in the evaluation: 89 test ratings, 44 of them relevant, so they lower recall instead of being dropped.

Ranking and metrics. Each model returns a top-10 list per user, with ties broken by item id, so results are deterministic. We report Precision@10 (hits / 10), Recall@10 (hits / relevant movies) and NDCG@10 with binary relevance, with the ideal list taken over min(relevant movies, 10) hits (Lecture 3.1, slides 7–8; see 1.2). These are averaged over users. Catalogue coverage is the number of distinct movies recommended to at least one user, divided by 3,883 (Lecture 3.1, slide 9). Our NDCG@10 matches scikit-learn's ndcg_score on 200 sampled test users (largest difference 2.2e-16).

Evaluated users. A user is evaluated in a period only if they have at least one earlier rating and at least one relevant rating in that period. Of the 1,209 users active in the test period, 29 are excluded as cold-start and 21 for having no relevant movie, leaving 1,159. Validation evaluates 492. Every model is scored on the same users, so results do not cover cold-start users (Lecture 1.2, slide 12), even though Most Popular could serve them.

Tuning. All model settings are selected on the validation period only, including for the baselines, which must be tuned like any other model (Lecture 3.1, slide 10). Each configuration is fit on train and scored on validation by NDCG@10. The selected configurations are then refit on train+val and evaluated once on test.

1.5 Baseline methods and settings

Item-based kNN (Lecture 1.2, slides 5–10) is implemented in src/item_knn.py. For each pair of items we compute a similarity from the user–item matrix. Each item keeps its k most similar items as its neighbourhood, and negative similarities are dropped. A user's score for item i is the sum, over the neighbours j of i that the user has rated, of sim(i,j) × the user's (transformed) rating of j. Items are ranked by score and ties are broken by item id, so results are deterministic.

We use the unnormalised sum rather than the rating-prediction formula on slide 5, which divides by the sum of |sim|. That division turns the score into a rating estimate, so an item reached through one weak neighbour can score as high as one reached through many strong ones. For top-N ranking this tends to push obscure items up, so we rank by the sum.

Settings evaluated.

- **Similarity:** five measures.
  - Binary cosine: every rating counts as 1.
  - Jaccard.
  - Cosine on raw 1–5 ratings.
  - Adjusted cosine: ratings minus the user's mean.
  - Item-centred Pearson: ratings minus the item's mean.
- **Neighbourhood size k:** 10, 20, 50, 100, 200, 400, 800, 1600, 3200. At 3200 there is almost no pruning, as the training data has 3,662 items.
- **Significance weighting (slide 10):** off, or γ=25, which multiplies the similarity by min (n_ij, 25)/25, where n_ij is the number of users who rated both items.

Cosine on raw ratings with k=1600 and γ=25, with validation NDCG@10 of 0.1345. Adjusted cosine with k=20 was a close second on validation (0.1320), and both are reported on test.

The scoring code matches a slow, literal implementation of the formula on random data, across all five similarities, two values of k and two γ settings. The maximum difference is about 1e-6 (tests/check_item_knn.py).

Most Popular (Lecture 1.1, slide 9) ranks movies by their number of ratings in the history data and gives everyone the same order, skipping movies they already rated. We evaluate four variants: all-time or a recent window, each counting either all ratings or only ratings of 4 or 5. The window length was chosen on validation by NDCG@10 from 1, 3, 7, 14, 30, 90 and 180 days, and 7 days was best for both. The selected variant is the one with the highest validation NDCG@10: last 7 days, ratings of 4 or 5 only (0.1385).

1.6 Results and analysis

The best item-based kNN (NDCG@10 of 0.222) is only about 0.01 above the best Most Popular variant on test (0.212, all-time, ratings ≥4). We have no confidence intervals yet, so we do not claim that kNN beats popularity. Part 2 will add paired tests across users.

The choice of similarity matters much more than significance weighting. Changing the similarity moves test NDCG@10 by up to 0.10, while significance weighting changes validation NDCG@10 by at most about 0.004 across the validation grid.

Raw-rating cosine and adjusted cosine are close on NDCG@10 (0.2222 against 0.2195) but behave differently. Adjusted cosine has higher recall (0.067 against 0.058) and about double the catalogue coverage (0.180 against 0.087), so it recommends a wider range of items.

Raw-rating cosine improves steadily as k grows up to 1600 and then dips at 3200. Adjusted cosine peaks at k=20 and is flat or non-monotone beyond that. We suspect that a large k lets popular items dominate the scores, because co-rating counts favour them, but we have not tested this.

Item-centred Pearson is clearly the worst on accuracy (0.125) but has the highest coverage (0.515). This is consistent with it ignoring popularity, and it illustrates the trade-off between accuracy and coverage.

Most Popular variants score close to each other on test NDCG@10 (0.201 to 0.212), which fits their similar top-10 lists: any two share at least half of their movies. The 7-day window with ratings of 4 or 5 uses only 6,642 ratings, against 520,829 for its all-time counterpart, yet reaches 0.210. This suggests that the most popular movies are easy to find from little data, because ratings are concentrated on few movies (see 1.1).

On test NDCG@10, three of the four variants score about 0.21, and only all-time with all ratings is lower (0.201). A 7-day window raises it to 0.210, and counting only ratings of 4 or 5 raises it to a similar 0.212, but in different ways: the window mainly raises recall (0.052 to 0.065), while the threshold mainly raises precision (0.185 to 0.195). These differences may be noise, as we have no confidence intervals.

On validation NDCG@10, all-time with all ratings is also lowest (0.1185), but the 7-day window raises it more (0.1365) than the threshold does (0.1261). One hypothesis, which we have not tested, is that the window matters more on validation because validation directly follows it, so recent popularity is still up to date, while test runs for two years.

Coverage is only 2.8% to 3.8% because nearly everyone receives the same list, so the selected kNN (NDCG@10 0.222, coverage 8.7%) mainly adds breadth over the selected Most Popular variant (0.210, 3.2%).

Validation NDCG@10 is much lower than test NDCG@10 for both selected baselines (0.135 against 0.222 for kNN, 0.139 against 0.210 for Most Popular), which fits the differences between the two periods described in

1.3. The validation-best kNN similarity (raw-rating cosine) is also the test-best, by 0.003 over adjusted cosine, while the validation-best Most Popular variant (7-day, ratings of 4 or 5) is 0.002 below the test-best (all-time, ratings of 4 or 5). Both margins are small and we have not tested whether they are real.

2. Part 2 - Models and Honest Comparison

2.1 Models

Describe the additional models and paradigms. Clearly identify the self-implemented algorithm and explain its implementation.

2.2 Hyperparameter tuning and variability

Describe the validation-only tuning procedure, search spaces/budgets, random seeds, and confidence intervals or statistical testing.

2.3 Overall results

Present and discuss the main comparison. Explain important differences and avoid unsupported claims.

2.4 Disaggregated analysis

Report and interpret results broken down by at least two of the required dimensions: items with few vs. many training interactions, low- vs. high-activity users, popular vs. long-tail items, and early vs. late test period. Define any thresholds used and focus on findings that change or qualify the overall conclusions.

2.5 Sparsity ablation

Report and discuss the 100%, 50%, and 20% training-interaction experiments, including changes in the ranking between models.

3. Part 3 - Beyond Accuracy

3.1 Selected track and motivation

State Track A, B, or C and briefly motivate the selected intervention/extension.

3.2 Method

Describe only your selected track. The Track A/B/C guidance below is provided as instructions; delete the guidance for the two tracks you did not select before submission.

If Track A - Diversity and popularity bias

State the base recommender, selected re-ranking method, genre-vector item representation, trade-off parameter, and exact item-distance measure. Describe how the parameter was swept.

If Track B - Fairness

Define the release-decade provider proxy, report and justify the final decade boundaries and group sizes, state and justify the adopted fairness notion, describe the consumer groups based on MovieLens user attributes, and identify the stage and form of the mitigation.

If Track C - Group recommendations

State the selected synthetic group type and justify the chosen group size (3-5). Describe the three aggregation strategies. Identify which strategy was extended to the sequential scenario and explain how satisfaction is carried forward between recommendation rounds.

3.3 Results and trade-offs

Present the required track-specific results. Track A: report intra-list diversity, catalogue coverage, Gini and calibration error across the parameter sweep, and discuss the accuracy-diversity frontier. Track B: report position-weighted exposure against catalogue share, per-group consumer-side accuracy, and the fairness-performance frontier. Track C: compare group satisfaction, disagreement and minimum member satisfaction across the three aggregation strategies, then analyse how member and group satisfaction develop in the sequential scenario.

3.4 Cost and interpretation

State what is gained, what is lost, and who bears the cost where applicable. Explain what operating choice or strategy you would select and why.

3.5 "One number we cannot explain"

Identify one result you do not fully understand and describe how you would investigate it.

4. Discussion and Conclusions

Summarize the main findings across the project. Discuss limitations, failed approaches where relevant, and what the experiments support.

5. Contribution Statement

Liisa did data characterization split and most common model. Documentation related to that was done by Liisa.

Jaan and Piitu did the item-based kNN and the documentation on it for part 1.

6. Use of AI Assistants

Claude Code (Opus 5.5) was used to generate code, analyze results and help with writing. We take responsibility that we understand and agree with all content AI created or assisted in creating.

References

F. Maxwell Harper and Joseph A. Konstan. 2015. The MovieLens Datasets: History and Context. ACM Transactions on Interactive Intelligent Systems 5, 4, Article 19.